

BLE Advertising Spec — Synchroone Sensor

Audiência: Desenvolvedor do firmware (STM32WBA65).

Objetivo: Definir o payload de advertising que permite ao gateway Cassia X2000 detectar automaticamente quando um sensor tem dados prontos para coleta, sem nenhum processamento adicional no serviço de borda.

1. Conceito Central

O sinal "tenho dados prontos" **não é um bit dentro do payload** — é a **presença ou ausência do bloco Manufacturer Specific Data** (AD Type 0xFF) no advertising packet.

Estado do sensor	AD Type 0xFF no adv	Gateway recebe evento?
Idle (sem dados)	ausente	Não — filtrado no gateway
Dados prontos para coleta	presente	Sim — 1 evento enviado

Isso significa que o gateway Cassia filtra sensores idle com zero custo computacional no serviço de borda. O sensor "aparece" no gateway apenas quando tem dados, e "desaparece" quando termina a transmissão.

2. Dois Estados de Advertising

Estado IDLE — sensor dormindo entre leituras

O payload **não contém** AD Type 0xFF :

[Flags][Local Name "Sync"]

Byte	Valor	Significado
0	0x02	length
1	0x01	AD Type: Flags
2	0x06	flags value (LE General Discoverable + BR/EDR not supported)
3	0x05	length
4	0x09	AD Type: Complete Local Name
5-8	Sync	"Sync" em ASCII

Total: **9 bytes**. Cabe folgado nos 31 bytes do payload primário.

Estado READY — dados prontos, aguardando conexão

O payload **contém** AD Type 0xFF após o nome:

[Flags][Local Name "Sync"][Manufacturer Specific Data]

Byte	Valor	Significado
0	0x02	length
1	0x01	AD Type: Flags

2	0x06	flags value
3	0x05	length
4	0x09	AD Type: Complete Local Name
5–8	Sync	"Sync" em ASCII
9	0x06	length = 6 (bytes após este campo)
10	0xFF	AD Type: Manufacturer Specific Data
11	0xFF	Company ID byte baixo (0xFFFF em desenvolvimento)
12	0xFF	Company ID byte alto (registrar no Bluetooth SIG para produção)
13	seq	seq_num – ver seção 3
14	status	status byte – ver seção 3
15	count	pending_count – ver seção 3

Total: **16 bytes**.

3. Campos do Manufacturer Specific Data

3.1 seq_num (byte 13) — uint8, incrementa a cada nova leitura

É o campo mais importante. Tem duas funções:

Sinaliza nova leitura ao gateway:

O filtro do Cassia usa `filter_duplicates=0` — ele suprime eventos de um mesmo dispositivo enquanto o payload não mudar. Quando o sensor gera uma nova leitura e incrementa o `seq_num`, o payload muda e o Cassia envia um novo evento, notificando o gateway que há um dado diferente disponível.

Deduplicação no serviço de borda:

O gateway anota o `seq_num` de cada leitura coletada com sucesso. Se o sensor re-anunciar com o mesmo `seq_num` após uma tentativa falha de conexão, o gateway reconhece que é o mesmo dado e reconecta. Se o `seq_num` for diferente do último coletado, é uma leitura nova.

seq_num é uint8 — faz wrap em 255. O gateway trata a comparação como `seq_num != último_coletado`, não como "maior que". Não resetar para 0 manualmente; deixar o wrap natural do uint8.

3.2 status (byte 14) — uint8, bitfield

Bit	Nome	Descrição
0	error_flag	1 = sensor em estado de falha. Gateway conecta mesmo assim e loga o evento.
1	low_battery	1 = bateria crítica. Gateway conecta e loga o alerta.
2–7	reservado	Manter em 0.

Exemplos:

- `0x00` — tudo normal
- `0x01` — erro ativo
- `0x02` — bateria crítica
- `0x03` — erro + bateria crítica

3.3 pending_count (byte 15) — uint8

Número de leituras completas acumuladas prontas para transmissão. Na implementação mais simples (sem pool em flash no device), este valor é sempre 1. Se o device tiver pool interno (múltiplas janelas de amostragem armazenadas), usar o contador real.

O gateway usa este valor para **priorizar** sensores com mais dados na fila de conexão — não para decidir se conecta ou não.

4. Implementação no Firmware

Localizar em STM32_WPAN/App/app_ble.c :

```
/* Company ID – usar 0xFFFF em desenvolvimento.
   Registrar no Bluetooth SIG para produção: bluetooth.com/specifications/assigned-
   numbers/ */
#define SYNC_COMPANY_ID_L 0xFF
#define SYNC_COMPANY_ID_H 0xFF

/* Advertising idle: AD Type 0xFF ausente – sensor invisível ao filtro Cassia */
static const uint8_t a_AdvData_Idle[] = {
    0x02, 0x01, 0x06,
    0x05, 0x09, 'S', 'y', 'n', 'c',
};

/* Advertising ready: AD Type 0xFF presente – sensor visível ao filtro Cassia */
static uint8_t a_AdvData_Ready[] = {
    0x02, 0x01, 0x06,
    0x05, 0x09, 'S', 'y', 'n', 'c',
    0x06, 0xFF, SYNC_COMPANY_ID_L, SYNC_COMPANY_ID_H,
    0x00, /* [idx 12] seq_num */
    0x00, /* [idx 13] status */
    0x00, /* [idx 14] pending_count */
};

#define ADV_READY_IDX_SEQ 12
#define ADV_READY_IDX_STATUS 13
#define ADV_READY_IDX_PENDING 14

static uint8_t seqNum = 0;

/**
 * Chamar após leitura do sensor concluída, antes de aguardar conexão.
 * Adiciona AD Type 0xFF ao payload – sensor torna-se visível no filtro Cassia.
 *
 * @param pendingCount Leituras prontas (normalmente 1).
 * @param status Bitfield de status: bit0=error, bit1=low_battery.
 */
void SyncAdv_SetReady(uint8_t pendingCount, uint8_t status)
{
    a_AdvData_Ready[ADV_READY_IDX_SEQ] = ++seqNum;
    a_AdvData_Ready[ADV_READY_IDX_STATUS] = status;
    a_AdvData_Ready[ADV_READY_IDX_PENDING] = pendingCount;
}
```

```

    aci_gap_update_adv_data(sizeof(a_AdvData_Ready), a_AdvData_Ready);
}

/**
 * Chamar após TX completo (handleStateDone).
 * Remove AD Type 0xFF – sensor some do filtro Cassia automaticamente.
 */
void SyncAdv_SetIdle(void)
{
    aci_gap_update_adv_data(sizeof(a_AdvData_Idle), a_AdvData_Idle);
}

```

5. Onde Chamar nas Funções de Controle

Em `Synchroone/synchroone_routine.c` :

```

/* Após leitura do sensor concluída, antes de aguardar conexão do gateway */
void handleStateStart(void)
{
    // ... leitura do IIS3DWB ...

    SyncAdv_SetReady(1, 0x00); // pendingCount=1, sem erros

    /* Advertising rápido para o gateway detectar em até ~50 ms */
    APP_BLE_Procedure_Gap_Peripheral(PROC_GAP_PERIPH_ADVERTISE_START_FAST);
}

/* Após transmissão concluída */
static inline void handleStateDone(void)
{
    UTIL_TIMER_Stop(&taskFlowCtrlTimerBurst);

    BLE_SESSION_DisconnectAll();

    SyncAdv_SetIdle(); // remove AD 0xFF – sensor some do filtro Cassia

    /* Advertising lento – sensor está disponível mas sem dados para coletar */
    APP_BLE_Procedure_Gap_Peripheral(PROC_GAP_PERIPH_ADVERTISE_START_LP);
}

```

6. Como o Filtro Cassia Funciona

O gateway Cassia X2000 recebe todos os advertising packets do ambiente via scan passivo. O serviço de borda Python se conecta ao endpoint SSE com os seguintes filtros:

```

GET /gap/nodes?event=1
    &filter_name=Sync
    &filter_manufacturer_data=FFFF

```

&filter_duplicates=0
&filter_rssi=-90

Filtro	Comportamento
filter_name=Sync	Descarta todos os dispositivos BLE do ambiente que não tenham "Sync" no nome. Elimina smartphones, beacons de terceiros, etc.
filter_manufacturer_data=FFFF	Só passa eventos onde o payload contém AD Type 0xFF começando com FF FF (Company ID 0xFFFF). Sensores idle não têm esse bloco — são descartados automaticamente.
filter_duplicates=0	Suprime eventos repetidos do mesmo dispositivo enquanto o payload não mudar. O sensor pode re-anunciar em 50 ms mas o gateway só encaminha 1 evento. Quando o seq_num muda (nova leitura), o payload muda e o gateway encaminha um novo evento.
filter_rssi=-90	Descarta sensores com sinal muito fraco para fazer TX completo.

Fluxo completo de um ciclo de coleta

Sensor	Cassia X2000	Serviço Python
-- idle adv (sem AD 0xFF) ----->	filtrado – descartado	
-- idle adv (sem AD 0xFF) ----->	filtrado – descartado	
[leitura IIS3DWB concluída]		
-- SyncAdv_SetReady(1, 0x00) ---->		
adv com AD 0xFF, seq=N	payload NOVO	
	-- SSE event, seq=N ----->	
		[conecta via POST
/gap/connect]		
<-- conexão estabelecida -----	-----	
-- TX: START→TIMESTAMP→dados→END		
	-- GATT SSE events ----->	
		[assembler coleta TX
completo]		
<-- disconnect -----	-----	
-- SyncAdv_SetIdle() ----->		
adv sem AD 0xFF	filtrado – descartado	
-- idle adv (sem AD 0xFF) ----->	filtrado – descartado	

O serviço Python recebe exatamente **1 evento SSE por leitura disponível**, no momento em que o dado fica pronto. Nenhum processamento de filtragem é necessário no código Python — o Cassia faz tudo.

7. Resumo das Tarefas de Firmware

#	Tarefa	Arquivo
---	--------	---------

F1	Criar a_AdvData_Idle[] (9 bytes, sem AD 0xFF)	STM32_WPAN/App/app_ble.c
F2	Criar a_AdvData_Ready[] (16 bytes, com AD 0xFF)	STM32_WPAN/App/app_ble.c
F3	Implementar SyncAdv_SetReady(pendingCount, status)	STM32_WPAN/App/app_ble.c
F4	Implementar SyncAdv_SetIdle()	STM32_WPAN/App/app_ble.c
F5	Chamar SyncAdv_SetReady(1, 0) em handleStateStart() antes de aguardar conexão	Synchroone/synchroone_routine.c
F6	Chamar SyncAdv_SetIdle() em handleStateDone() após desconectar	Synchroone/synchroone_routine.c

8. Referências

- **BLE Spec 5.4 — AD Types:** Manufacturer Specific Data = 0xFF ; Company ID registry: bluetooth.com/specifications/assigned-numbers/
- **STM32WBA65:** aci_gap_update_adv_data() — atualiza o payload de advertising sem interromper o advertising em curso.
- **Cassia X2000 REST API:** /gap/nodes — endpoint SSE de scan com suporte a filter_manufacturer_data , filter_name , filter_duplicates , filter_rssi .